

CDA 4205L Lab #5: Floating Point Arithmetic

University of South Florida

Report due: One week after the lab section

Notes: Demonstrate Tasks 1 and 2 (Steps 1-4) to your TA during the lab session (30%). Answer the remaining questions and submit your report with all required elements by the deadline (70%).

Welcome to CDA 4205L Lab #5! The goal of this lab is to help you understand how floating point arithmetic instructions work and how they are used in a RISC-V assembly program.

Prelab

Floating Point Instructions

In class, you will cover how floating point values are *encoded* in a specific way that differs from standard binary. Indeed, the usual arithmetic operations that work for binary will not work for floating point. Special hardware components process floating point arithmetic, and floating point values are stored in special registers, separate from the hardware and registers used in integer arithmetic.

There are special instructions to convert a binary value to a single or double-encoded binary string. The conversion must be done before the value can be processed by the floating point hardware. The destination register of the conversion instruction is a floating point register. The inputs to the floating point arithmetic instructions must be floating point registers.

For example, we can load the number 100 into the integer register `t0`, then convert it to single precision floating point and store the converted value into the floating point register `ft0` as follows:

```
li      t0,    100
fcvt.s.w ft0,  t0
```

Instructions for addition, multiplication, and so on are generally of the format `f<instr>.s/d` where `<instr>` is the arithmetic instruction (e.g. `fadd`), and the `.s` or `.d` designation indicates whether it should operate on single or double-precision values.

Some other useful instructions for this lab include floating-point comparisons, which take the form:

```
f<gt/lt/ge/le>.s/d <integer reg>, <float 0>, <float 1>.
```

For example, `flt.d t0, ft0, ft2` will set `t0 = 1` if `ft0 < ft1`, else `t0 = 0`.

For a complete listing of floating point instructions and their operation, refer to the RARS manual <https://github.com/TheThirdOne/rars/wiki/Supported-Instructions>.

Newton-Raphson Method

In the lab, you must write a RISC-V assembly program that computes square roots both with and without using dedicated square root instructions. To find square roots *without* using a square root instruction, one option is to use numerical methods to approximate the square root. One such method is called Newton-Raphson. This is a general technique for finding the roots of a function. The general form for this method is

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

The basic idea is to find the root of a function $f(x)$ by approximating it with its derivative. In the equation, x_n is the current “guess” for the square root, and x_{n+1} is a closer guess. If we want the square root of 4205, $f(x)$ is $x^2 - 4205$, and our derivative $f'(x) = 2x$.

In Step 1, we pick an initial guess, e.g. $4205/10 = 420.5$, and solve the equation

$$x_{n+1} = 420.5 - \frac{420.5^2 - 4205}{2 * 420.5}$$

which gives our next guess of $x_{n+1} = 215.25$. Repeating this gives

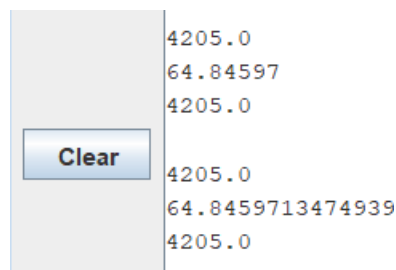
$$x_{n+2} = 215.25 - \frac{215.25^2 - 4205}{2 * 215.25}$$

and the next guess of 117.393, then 76.606, 65.749, 64.852, and finally 64.846... So, after a few iterations, we have an accurate estimate of the (positive) square root. The more iterations, the closer you will get. You can either have a fixed number or loop until the change in the value from one iteration to the next is less than some threshold, e.g. 0.00001.

Lab

1. Download the `nr-method-template.asm` file from Canvas. This contains some routines for printing floats, doubles, newlines, etc. and has comments indicating where to write your code.
2. Begin by loading a value into `t0` (one is already there, but you should change it). Your value should not be a perfect square. Perform the following:
 - a. Convert the value to a **single-precision** floating point. Store the result in `fa0` (floating point argument register). Print the result.
 - b. Perform the square root operation on the conversion result. Print the result.
 - c. Square the result by multiplying it by itself, and print the result again.
 - d. Repeat steps 2a – 2c with a *double* precision floating point value.

Example output:



```
4205.0
64.84597
4205.0
4205.0
64.8459713474939
4205.0
```

T1: You may expect the result of `square(sqrt(value))` to return the original, however this may not always be the case in practice. Why?

3. Now, **without** using the built-in `fsqrt.d` instruction, implement a function that returns the approximate square root of a number using the **Newton-Raphson** method. Your function should take in the number of iterations as well as n (the number you want to find the square root of), and constants 2 and 10. The constants 2 and 10 are used to find the initial guess: if $n < 10$, the initial guess should be $n/2$; else, the initial guess should be $n/10$. The initial guess must be computed inside the function.
4. Test your function by calling it multiple times with 3, 5, 7, and 9 iterations (i). Upon returning, print the result to the console. For each result, compare it to the output of `fsqrt.d` (**you don't need to calculate this in code, just use a calculator**).

T2: Include a table showing the original value, the “guess” after 3, 5, 7, and 9 iterations, and the error compared to the output of `fsqrt.d` (which you computed in step 2d).

5. Rather than specifying the number of iterations, modify your code so that the loop exists only when the result of the next iteration differs by a small enough amount. For example, you could choose to loop until the current result is within 0.00001 of the previous result. So, if the current output is 64.845971**3475**, and the previous iteration was 64.845971**6436**, the difference is less than the threshold, so the computation is done.

T3: Pick a number between 10^5 and 10^6 and run your code while varying the threshold from 10^{-1} to 10^{-8} . Include a table that shows how many iterations it took before the loop exits for each of the 8 thresholds. Include the percent error between the calculated and actual values (as given by a calculator). The error should decrease. Besides that, what other trends do you observe as the threshold changes?

Submit your report pdf with answers to all questions and screenshots. Also, submit the final assembly file with your implementation of the Newton-Raphson method.